- From the previous scenario exercise, you provided with the AWS Cost Calculator and Auto-scaling, you have been asked by the organization to explain your recommendations for DR & Resiliency. You have been tasked to identify the following:
  - What is your recommended Recovery Time Objective (RTO) and Recovery Point Objective (RPO)?
  - What would be necessary for your project to ensure a solid DR plan?
  - What would you do to ensure your application is resilient?
  - Would you use certain managed services, deployment strategies or other precautions to avoid an outage?
  - What type of "chaos engineering" techniques would you use to test resiliency?
  - You have 15-20 minutes to discuss your recommended plan
- Download the template from Assignments > Activity - DR & Resiliency Recommendations
  - 1 submission per team
- The scenarios can be found in Assignment > Activity - Cost Estimating Scenarios

RECAP

- Which of the following best describes Recovery Point Objective (RPO)?
  A. The maximum amount of downtime an application can tolerate
  B. The maximum acceptable amount of data loss before business harm occurs ✅
  C. The speed at which new features can be deployed
  D. The number of concurrent users an application can handle
- Which disaster recovery strategy involves keeping a small, always-on core system that can quickly scale into a full environment when needed?
  A. Backup and Restore
  B. Pilot Light ✅
  C. Warm Standby
  D. Multi-site Solution
- Resiliency is the same as disaster recovery, as both focus only on restoring operations after failure
  - True
  - False ✅

- Downtime is inevitable, even if you have systems in the cloud
- Prevention is the key for avoiding unplanned downtime and reducing its impact
  - Planning for Disaster Recovery
  - Building Resilient applications
  - Having a Deployment Strategy that minimizes downtime
  - Thorough testing for potential cloud outage scenarios (e.g. chaos engineering)
- How much organizations are willing to spend on prevention is largely tied to their RPO and RTO:
  - Recovery Point Objective (RPO) Refers to the maximum acceptable amount of data loss an application can undergo before causing measurable harm to the business
  - Recovery Time Objective (RTO) States how much downtime an application experiences before there is a measurable business loss

Big Data Concepts: Hadoop & Spark Basics

Engineering Cloud Software Systems

Department of Software Engineering
Rochester Institute of Technology

Activity Setup

- First - Do Assignments > Big Data > Setup EMR
  - There is a small charge for this activity
  - Stop after slide #3 (which you click "Create Cluster"), you are done for now (no screen shots needed)
  - This will take several minutes to run, and we will return to this shortly

Consider this problem

- DynamoDB can support tables of virtually any size with horizontal scaling
- This enables DynamoDB to scale:
  - More than 10 trillion requests per day
  - Peaks greater than 20 million requests per second
  - Petabytes of data
- Question: what if had process the data before loading, or we had to extract and reprocess this data? How would you do it?

This is a "Big Data" problem

What do we mean by "Big Data"?

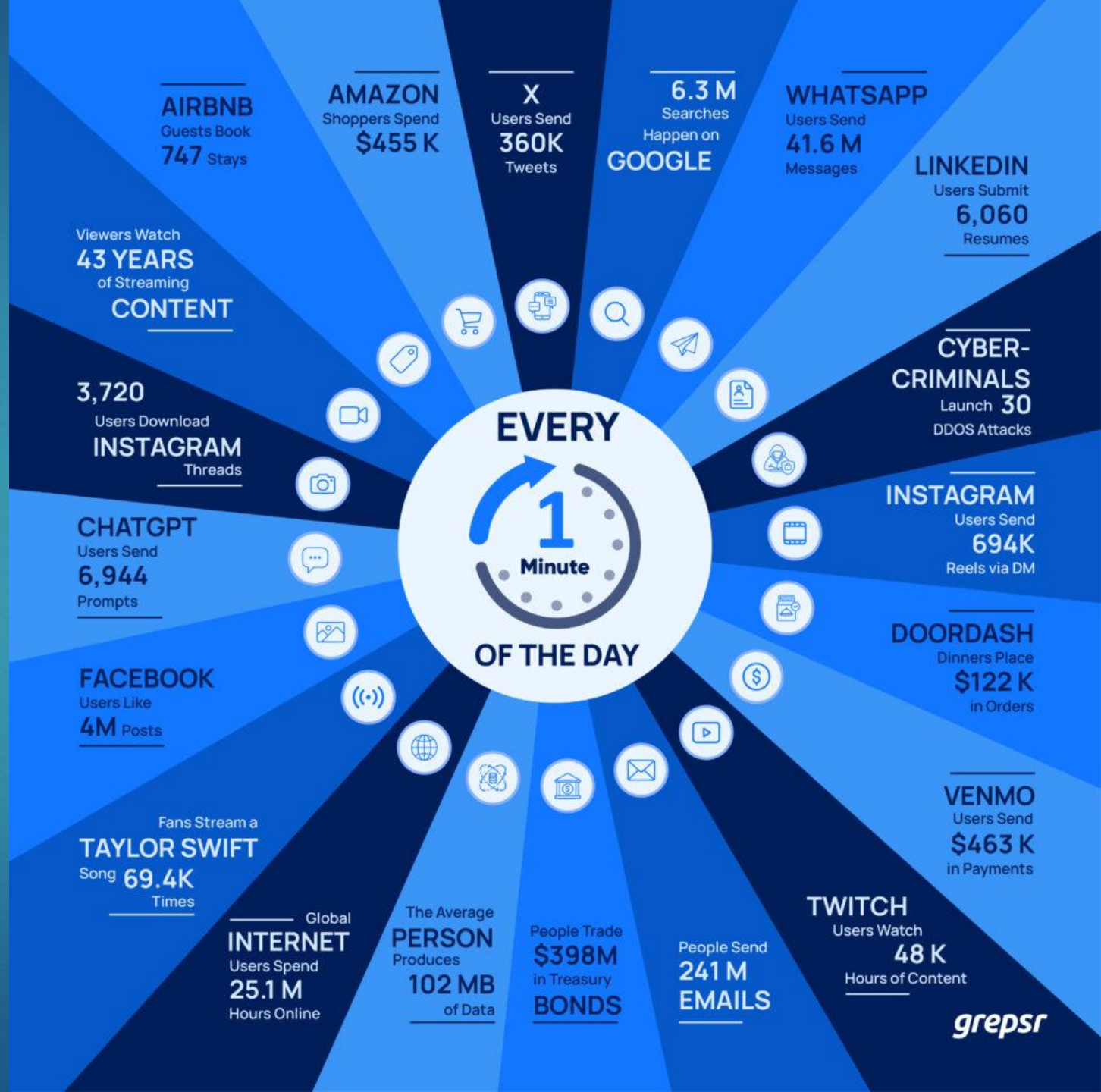
- Big Data is described by V's, to name a few:
  - **Volume** - Refers to the massive amounts of data generated every second from devices, sensors, social media, transactions, etc. (measured in terabytes, petabytes, or more)
  - **Velocity** - The speed at which data is created, transmitted, and processed (e.g., real-time streaming from IoT devices, stock markets, or social feeds)
  - **Variety** - The many forms data can take: structured (databases), semi-structured (JSON, XML), and unstructured (videos, images, text, audio)
  - **Veracity** – The quality, reliability, and accuracy of the data, accounting for noise, bias, or uncertainty

Source: https://medium.com/@igniobydigitate/learning-the-four-vs-of-big-data-can-lead-to-a-fifth-value-a1acd966ebdb

Big Data – Variety & Velocity

- This is only for 1 minute!
- 1 day ➔1440 minutes

<u>Where</u> can I process all this all this data?

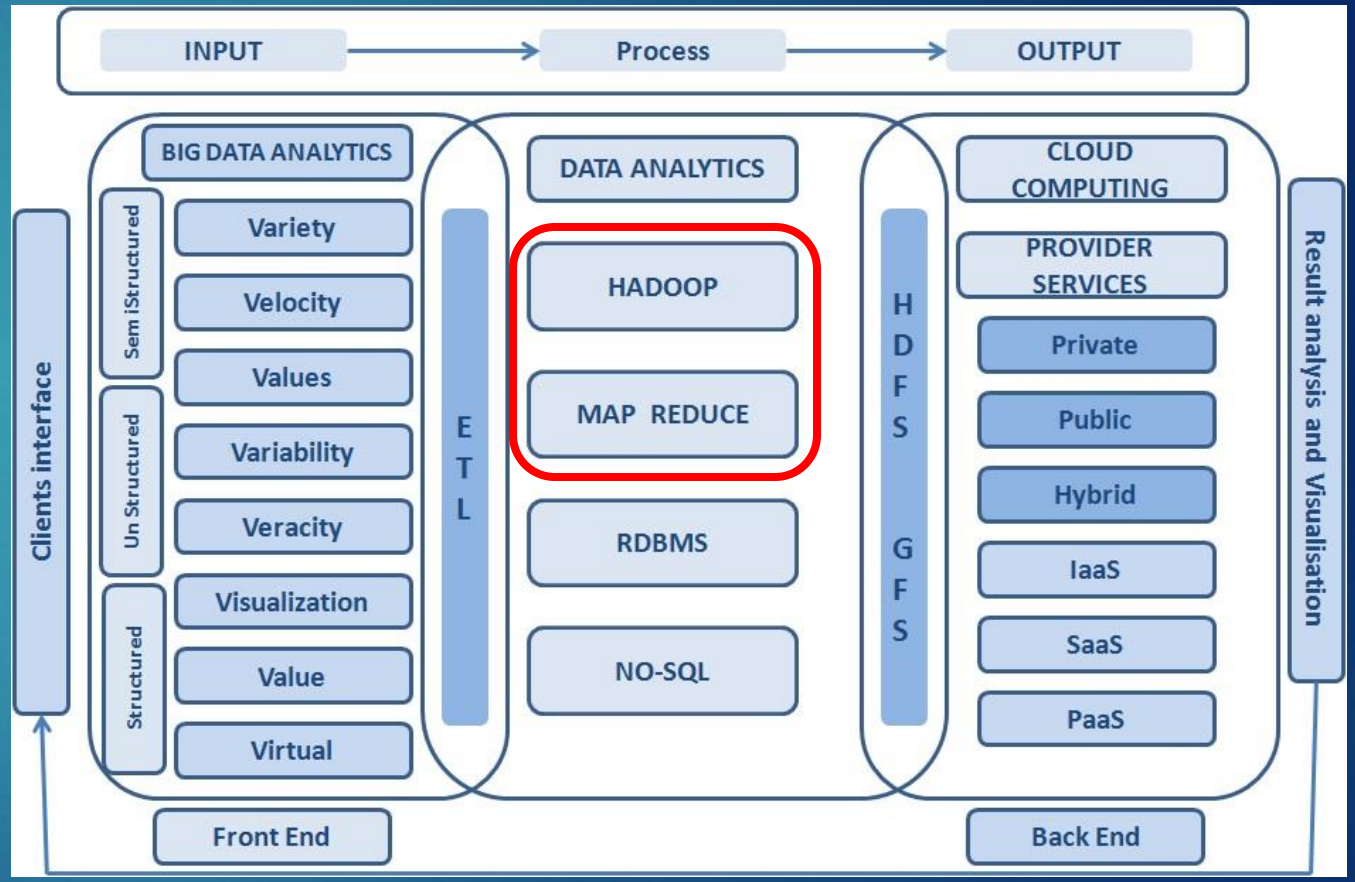


Big Data & Cloud Computing

- Big Data represents the **content** (large, complex datasets), while Cloud Computing provides the **infrastructure** to store, process, and analyze it
- Together, they deliver scalable, efficient, and cost-effective solutions
- Key Advantages:
  - **Agility** - Rapid provisioning of infrastructure and resources, accelerating development and deployment
  - **Elasticity** - Dynamic scaling to handle growing data volumes while maintaining performance
  - **Cost efficiency** - Pay-as-you-go model reduces infrastructure costs; providers manage hardware and operations
  - **Data processing** - Cloud platforms offer tools and scalability to process and analyze massive datasets quickly

Big Data & Cloud Computing – tying it all together

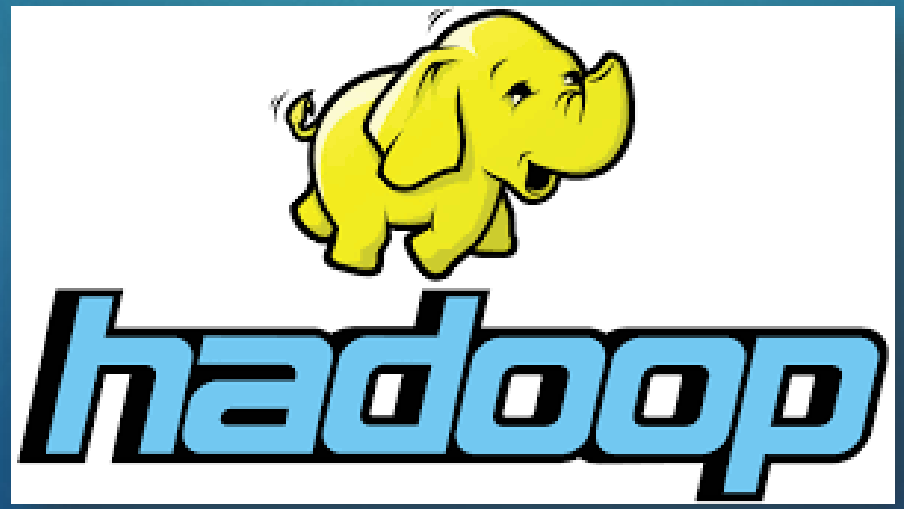
- Cloud computing provides scalable infrastructure and services to store, manage, and analyze massive datasets generated by Big Data
- Big Data drives the evolution of cloud services, pushing providers to deliver more powerful, flexible, and cost-efficient solutions
- Together, they amplify each other's value, driving innovation and deeper insights



Source: https://www.whizlabs.com/blog/big-data-and-cloud-computing/

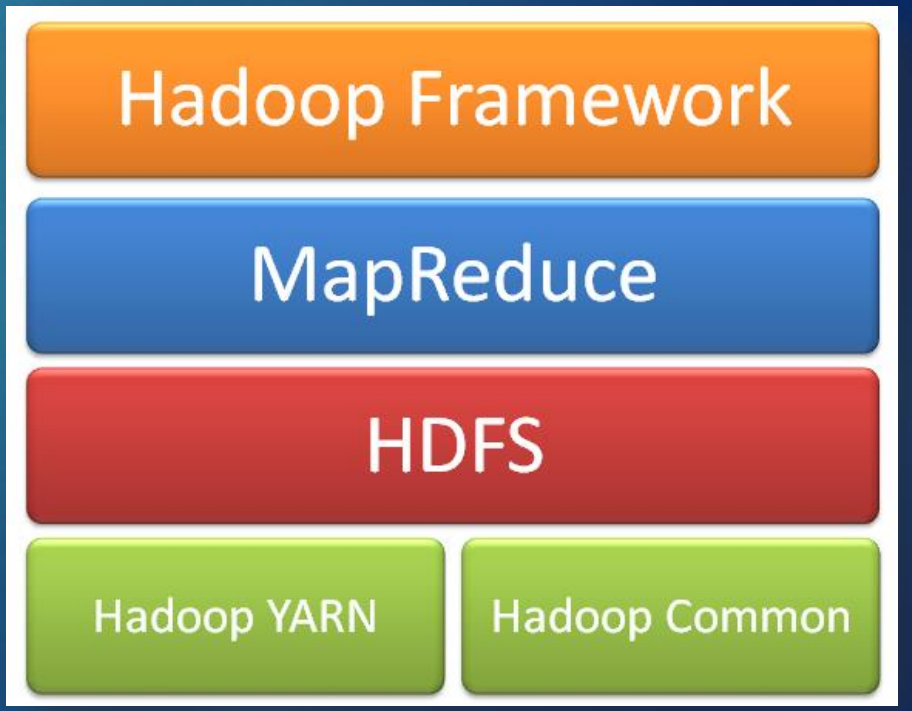
What's Hadoop?

- Hadoop is an open source, Java-based framework used for storing and processing Big Data
- It is designed to divide from single servers to thousands of "commodity" servers, each having local computation and storage
- It provides massive storage for any kind of data, enormous processing power and the ability to handle virtually limitless concurrent tasks or jobs



Hadoop Framework

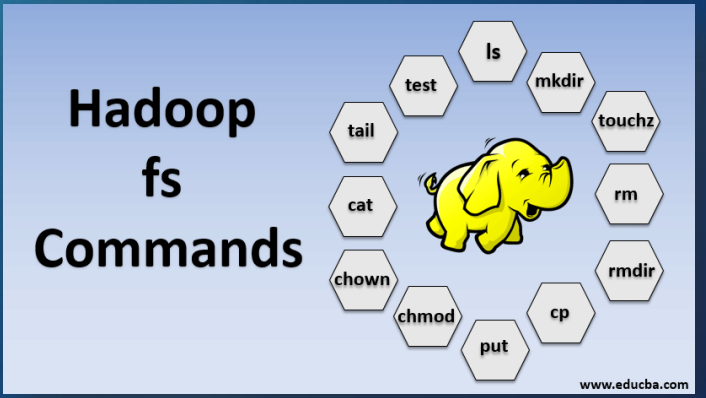
- Hadoop Common
  - Contains libraries and utilities needed by other Hadoop modules
- Hadoop Distributed File System (HDFS)
  - A distributed file-system that stores data on commodity machines, providing very high aggregate bandwidth across the cluster
  - Data on HDFS is immutable (you cannot overwrite)
- Hadoop YARN (Yet Another Resource Negotiator)
  - A platform responsible for managing computing resources in clusters and using them for scheduling users' applications
- Hadoop MapReduce
  - An implementation of the MapReduce programming model for large-scale data processing



HDFS commands

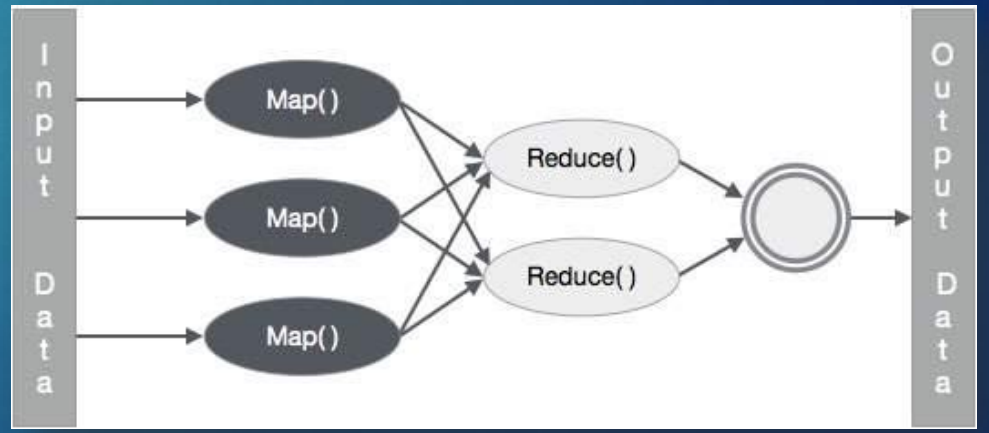
- HDFS commands resemble their Linux counterparts:
  - hadoop fs -ls *<directory>* — List the files in directory
  - hadoop fs -mkdir *<directory>* — Make a new directory
  - hadoop fs -cp *<from>* *<to>* — Copy data from one place to another
  - hadoop fs –rm –R *<directory>* — Remove directory and sub-directories
  - hadoop fs –rm *<file>* — Delete a file
  - hadoop fs -copyFromLocal *<file/folder>* — Copy data from local disk into cluster
  - hadoop fs -copyToLocal *<file/folder>* — Copy data from cluster to local disk

- Running hadoop fs with no parameters to see a list of all available commands
- You will need these for a future activity



What is MapReduce?

- MapReduce is an algorithm for processing Big Data sets in parallel
- It consists of two distinct tasks – Map and Reduce
- The first is the Map job, where a block of data is read and processed to produce key-value pairs as intermediate outputs
- The output of a Mapper or map job (key-value pairs) is input to the Reducer
- The Reducer receives the key-value pair from multiple map jobs
- The Reducer aggregates those intermediate data tuples (intermediate key-value pair) into a smaller set of tuples or key-value pairs which is the final output



MapReduce - Example

- You have been asked to count all the Apple and Android phones in this building
- There are 100 floors
- If you went to each floor, assume it takes 15 minutes per floor to count all the phones before you go to the next floor
- For 1 person to count all of the phones in the entire building floor by floor (in sequence) would then take:
  - 100 floors * 15 minutes/floor = 25 hours

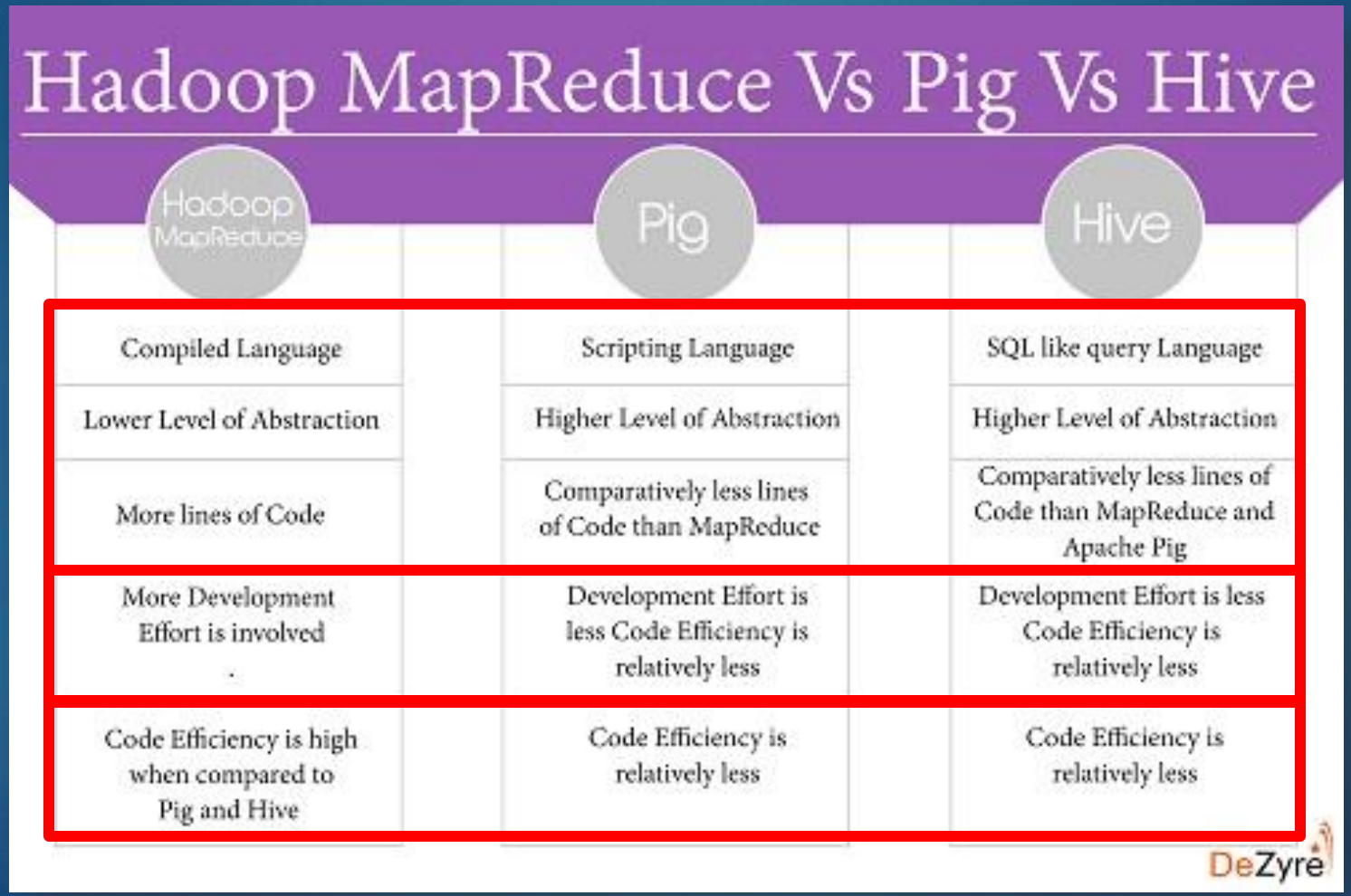
MapReduce - Example

- You have been asked to count all the Apple and Android phones in this building
- There are 100 floors
- Suppose you had 100 people and each person is assigned to count the phones on one of the 100 floors at the same time
  - Each person provides 2 key-value pairs ("Android": X) ("Apple": Y) where X and Y is the number of phones per floor
- Combine all key-values from all 100 people, we have:
  - ["Android": 20, "Apple": 30, "Android": 14, "Apple": 55, "Android": 10, "Apple": 43, …]
- This entire key-value list of 100 gets passed to 2 people:
  - One person totals the counts for "Android", the other for "Apple"
- Total time ~15 minutes (almost 100x faster)
- The 100 people are Mappers and the other 2 are Reducers

How do I write MapReduce?

- Write a MapReduce program in Java or Python
  - Complex to understand and maintain
- Apache Hive
  - "Relational database" built on Hadoop
  - You write SQL and it is compiled to MapReduce code
  - Proven to scale over 300 PB
- Pig
  - A simple scripting language for queries and data manipulation
  - It will consume any data that you feed it: structured, semi-structured, or unstructured
  - Like Hive, Pig is compiled to run as MapReduce jobs

MapReduce, Pig and Hive have their own pros and ons

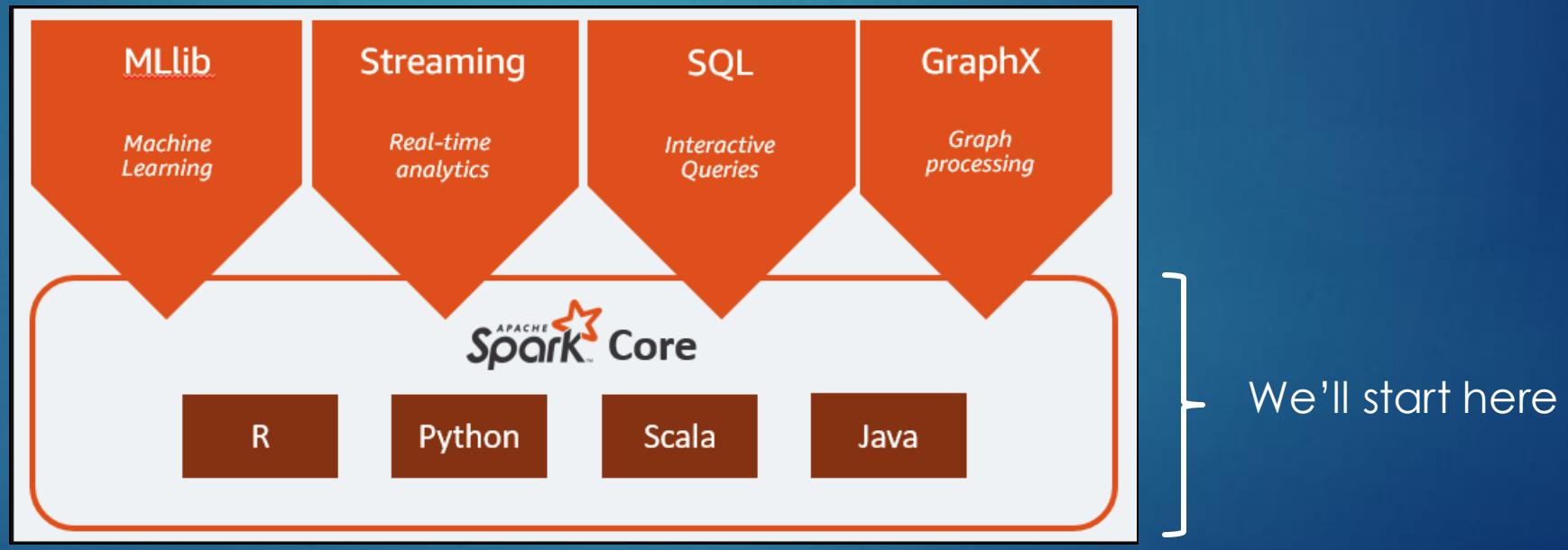


Is there an easier way?

Source https://www.dezyre.com/article/mapreduce-vs-pig-vs-hive/163

Introducing Apache Spark

- Apache Spark is a distributed, data processing engine for batch and streaming modes featuring SQL queries, graph processing, and machine learning
- You can program Spark in Java, Python, Scala, and R
- It can run in Hadoop clusters through YARN or in standalone mode



- Spark runs on AWS (EMR), Azure and Google Cloud

Spark Use Cases

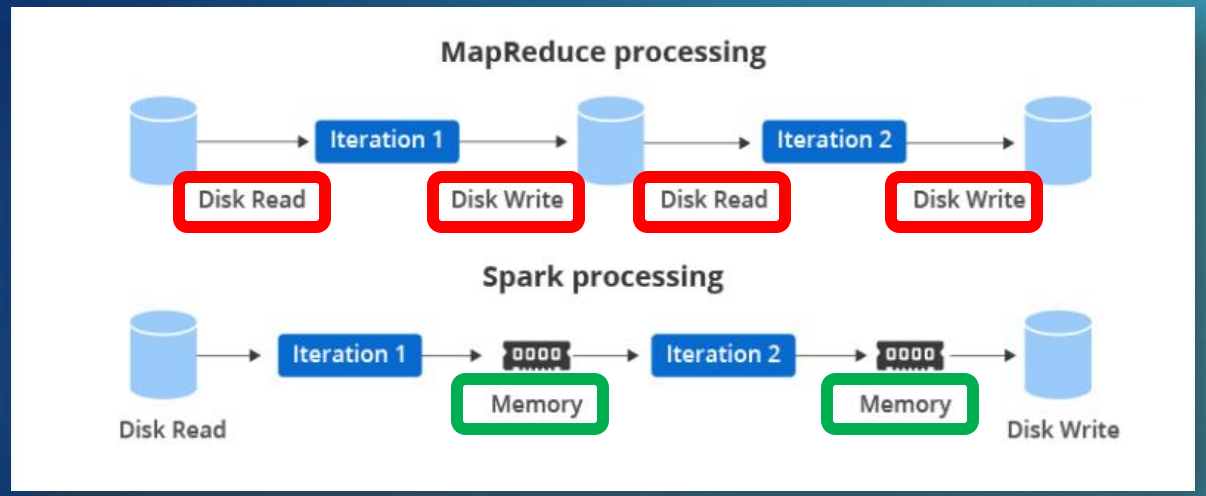
- Spark is mainly used for real-time data processing and time-consuming Big Data operations
- Since it's known for its high speed, itis in demand for projects that work with many data requests simultaneously

Companies that use Spark

Source: https://tharong.com/huong-dan-su-dung-apache-spark

Spark vs. Hadoop MapReduce

- Compared to MapReduce, Spark runs 10 -100x times faster due to in-memory processing



	Hadoop MR Record	Spark Record	Spark 1 PB
Data Size	102.5 TB	100 TB	1000 TB
Elapsed Time	72 mins	23 mins	234 mins
# Nodes	2100	206	190
# Cores	50400 physical	6592 virtualized	6080 virtualized
Cluster disk throughput	3150 GB/s (est.)	618 GB/s	570 GB/s
Sort Benchmark Daytona Rules	Yes	Yes	No
Network	dedicated data center, 10Gbps	virtualized (EC2) 10Gbps network	virtualized (EC2) 10Gbps network
Sort rate	**1.42 TB/min**	**4.27 TB/min**	**4.27 TB/min**
Sort rate/node	**0.67 GB/min**	**20.7 GB/min**	**22.5 GB/min**

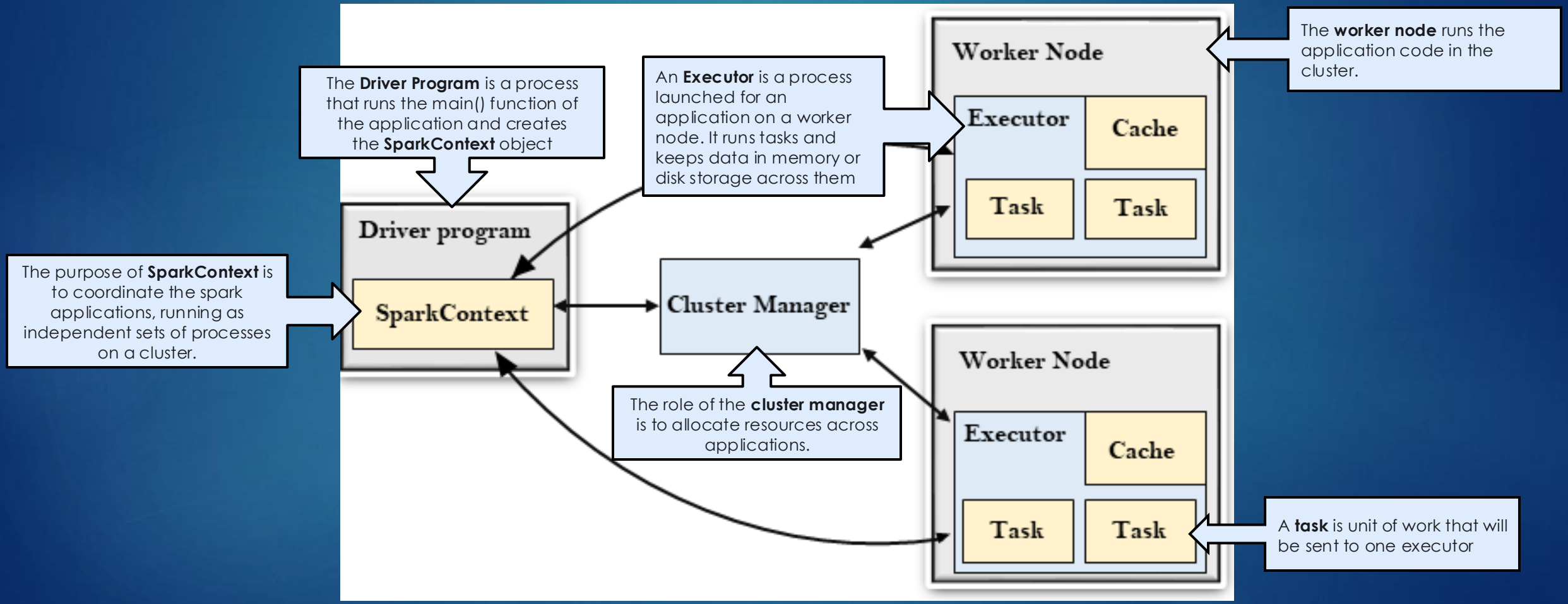
2014 Gray Sort competition, a 3rd-party benchmark measuring how fast a system can sort 100 TB of data (1 trillion records)

- Hadoop MapReduce ➔ involves more reading and writing from disk
- Spark ➔ executes much faster by caching data in memory across multiple parallel operations

Source: https://eduinpro.com/blog/apache-spark-vs-hadoop-mapreduce/

Spark Architecture

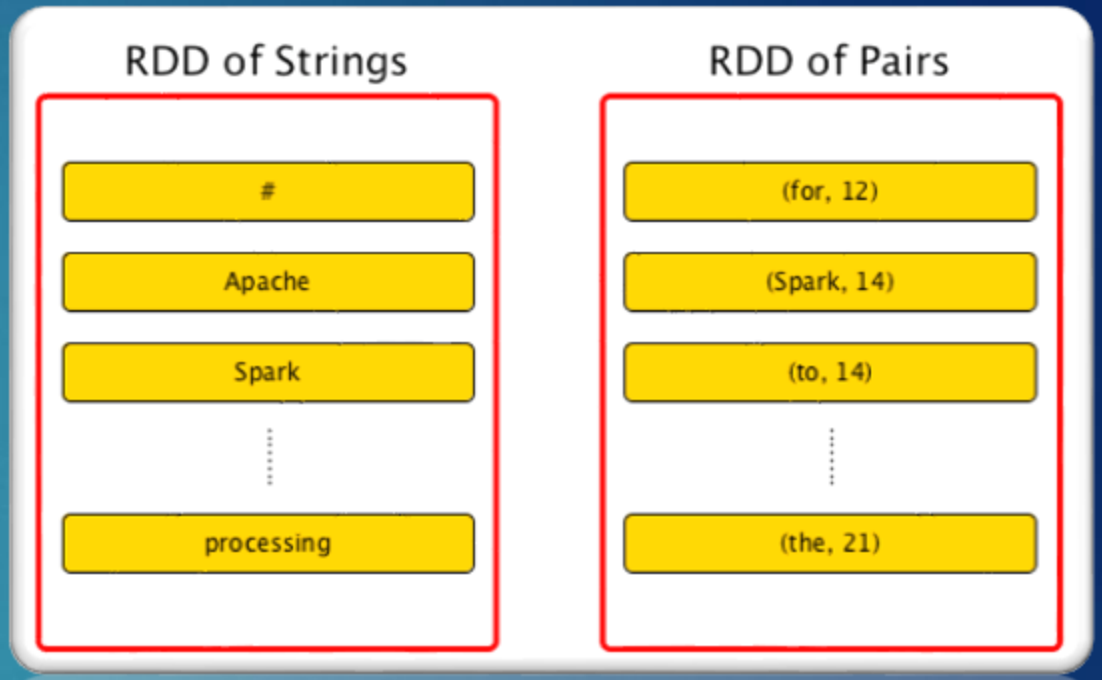
- Spark uses a master/worker architecture



Source: https://www.javatpoint.com/apache-spark-architecture

Spark Architecture - RDD

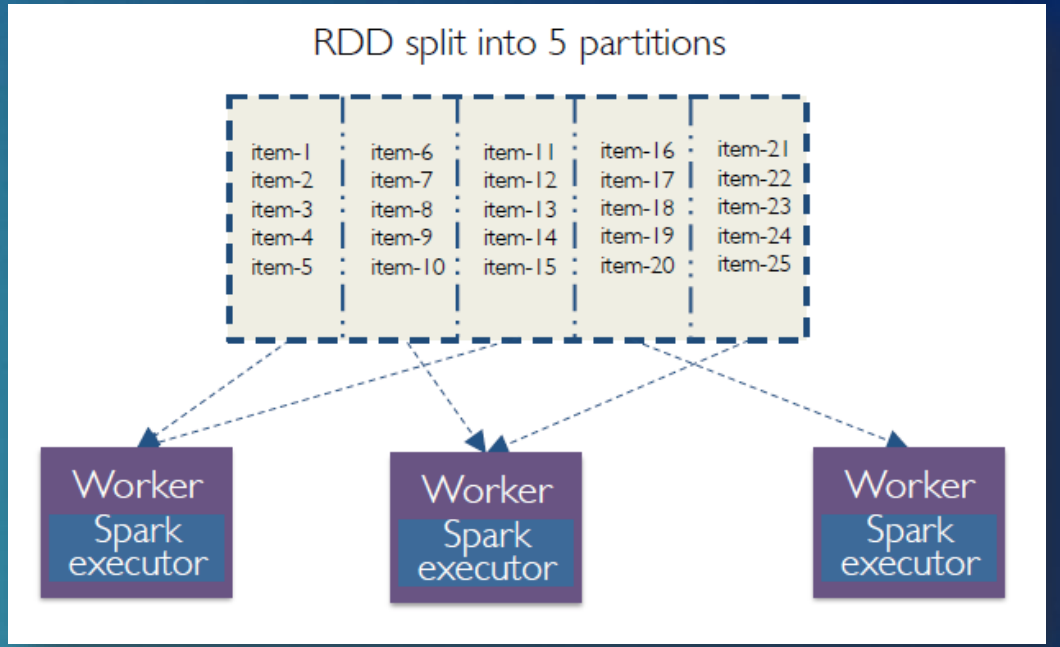
- RDDs Stands for:
  - **Resilient** - Fault tolerant and is capable of rebuilding data on failure
  - **Distributed** - Distributed data among the multiple nodes in a cluster
  - **Dataset** - Collection of partitioned data with values
- RDDs are the building blocks of any Spark application
- It is a read-only (i.e. immutable) distributed collection of objects



- Each dataset in RDD is divided into logical partitions, which may be computed on different nodes of the cluster
  - Why important - This allows for parallel processing

Spark Architecture - Partitions

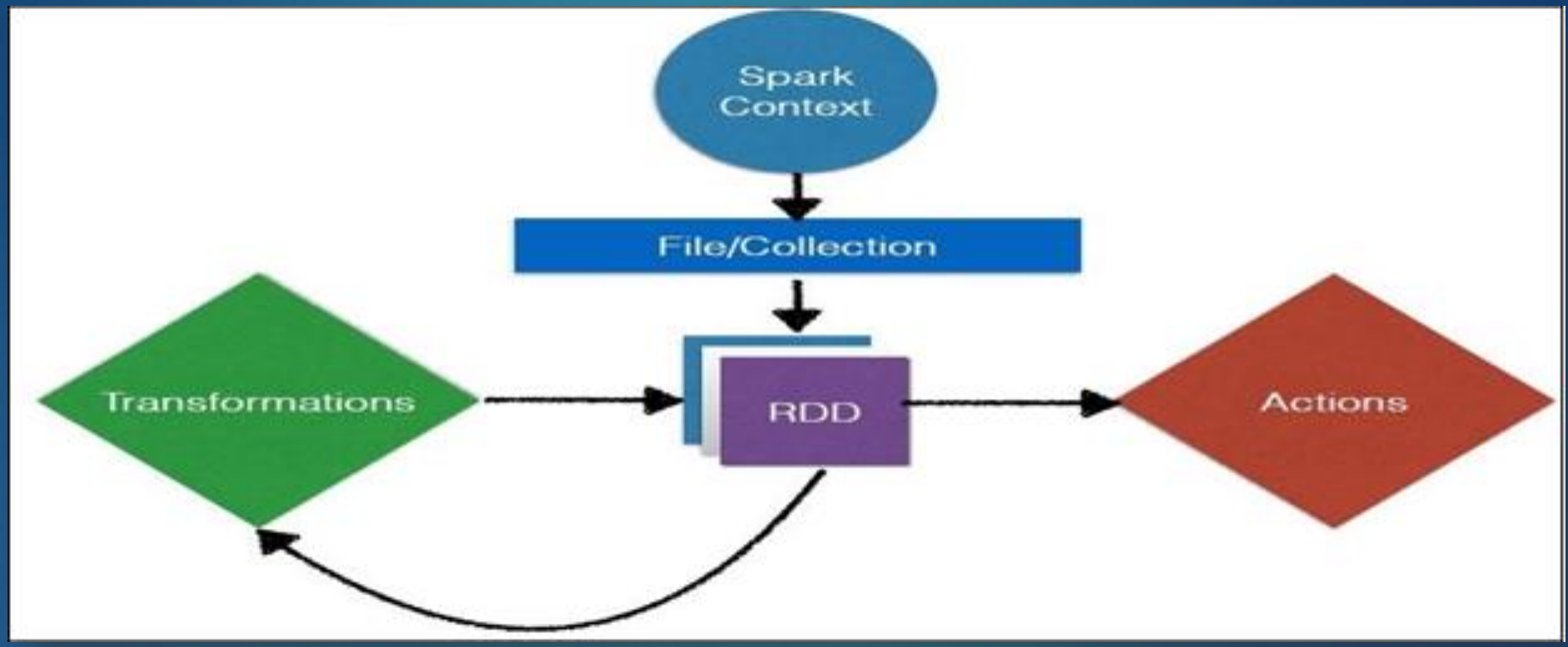
- RDDs are collections of objects
- Under the hood, RDDs objects are stored in partitions
- When performing computations on RDDs, these partitions can be operated on in parallel
- Understanding how Spark deals with partitions allow the developer to control the application parallelism which leads to better cluster utilization fewer costs and better execution time
- It's relatively easy to write a Spark program, but can be difficult to write one efficiently



Source: https://medium.com/@thejasbabu/spark-under-the-hood-partition-d386aaaa26b7

Spark RDD - Operations

- There are 2 types of operations for RDDs – Transformations and Actions

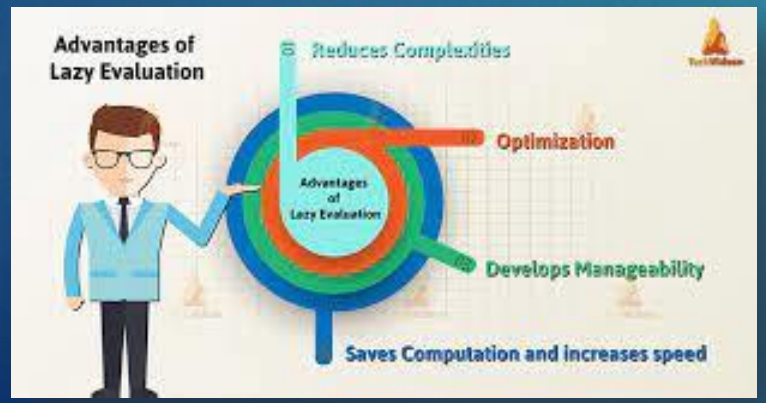


- Transformations create RDDs from each other
- Actions return some sort of result

Source: https://medium.com/@amoghdamle/rdd-transformation-and-actions-d7bf330d8a2b

Spark RDD - Transformations

- Transformations are functions that take an RDD as the input and produce one or many RDDs as the output
- They do not change the input RDD (since RDDs are immutable), but always produce one or more new RDDs by applying the computations they represent
- The are also "lazy", which means they do not compute a result until an Action is called



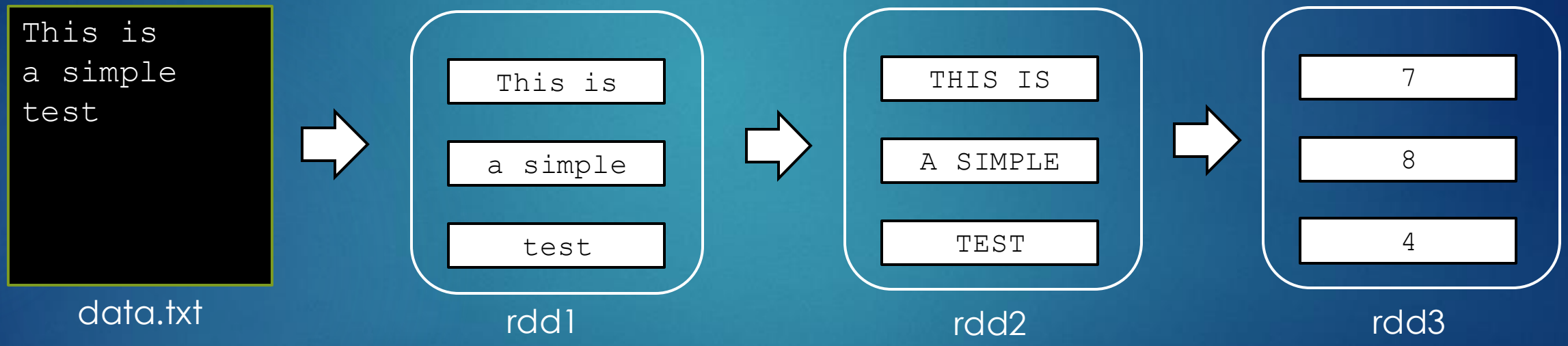
Transformations

map (func)
flatMap(func)
filter(func)
groupByKey()
reduceByKey(func)
mapValues(func)
sample(...)
union(other)
distinct()
sortByKey()
...

Source: https://medium.com/@amoghdamle/rdd-transformation-and-actions-d7bf330d8a2b

Spark – Example #1 - Map

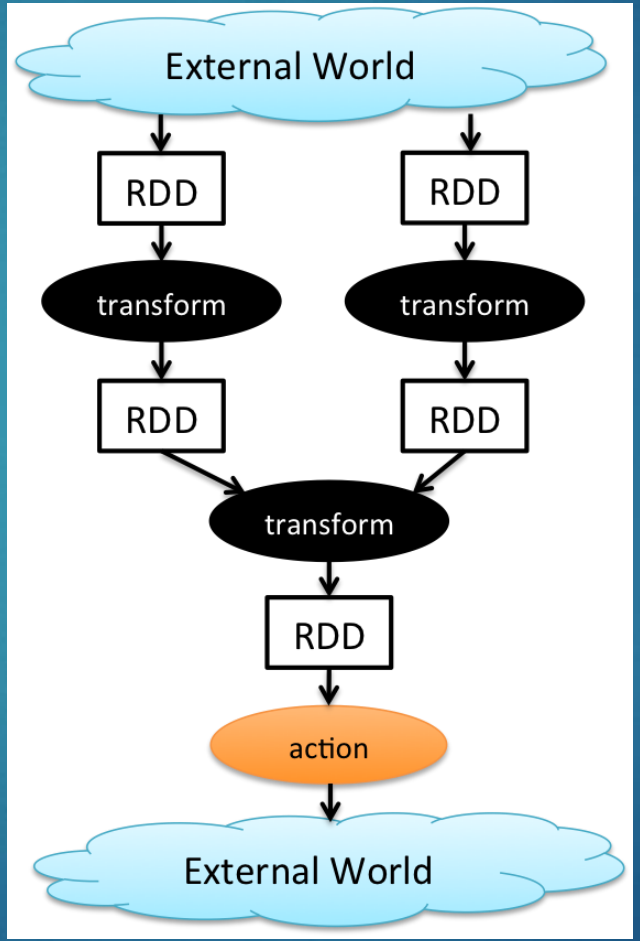
- A **map** is a transformation operation in Apache Spark
- It applies a function to each element of RDD and it returns the result as new RDD




```
rdd1 = sc.textFile("data.txt")          // SC = Spark Context
rdd2 = rdd1.map(lambda line: line.upper())
rdd3 = rdd2.map(lambda line: len(x))
```

Spark RDD - Actions

- Actions are operations to access the actual data available in an RDD and provide non-RDD values
- Transformation's output is an input of Actions
- It is one of the ways of sending data from Executer to the Spark driver
- They are not lazily executed



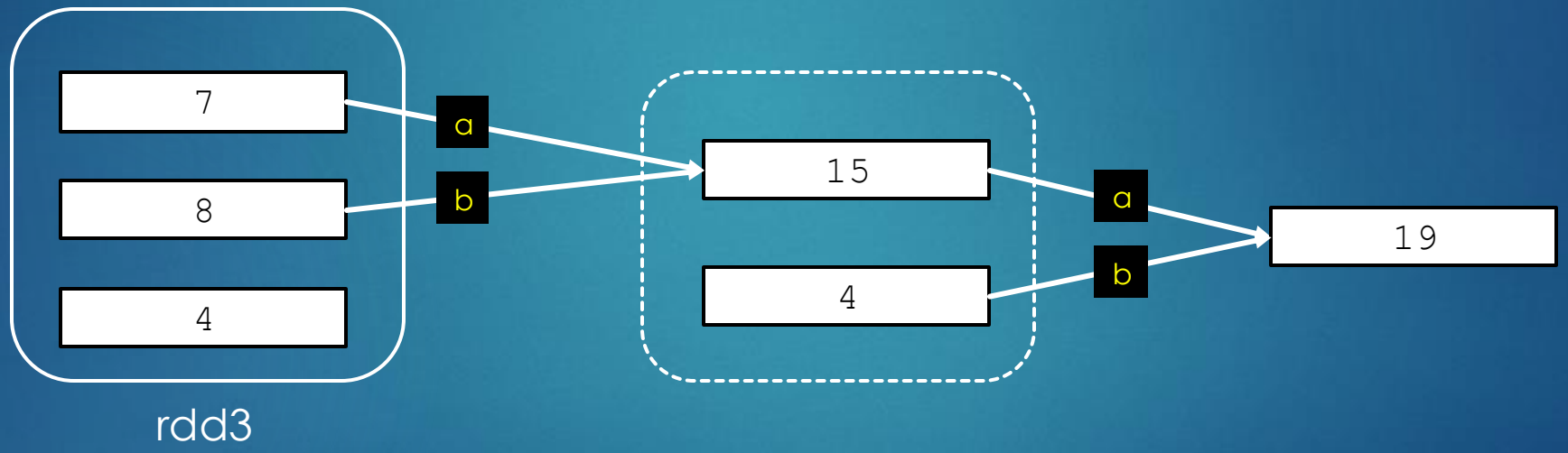
Actions

```
reduce(func)
collect()
count()
first()
take(n)
saveAsTextFile(path)
countByKey()
foreach(func)
...
```

Source: https://medium.com/@amoghdamle/rdd-transformation-and-actions-d7bf330d8a2b

Spark – Example #1 – Map + Reduce

- A **reduce** is an action that aggregates an RDD element using a function
- That function takes two arguments and returns one




```
rdd1 = sc.textFile("data.txt")        // SC = Spark Context
rdd2 = rdd1.map(lambda line: line.upper())
rdd3 = rdd2.map(lambda line: len(x))
totalLength = rdd3.reduce(lambda a, b: a + b) // Answer 19
```

Spark – Example #2

- How would you describe the steps to count all the words?
  1. Split each word at the space
  2. For each word, create a key (word) and count (1)
  3. Count all words grouped by their key

```
w1  w2  w3
w1  w1  w3
w1  w3  w3
```

Spark – Example #2


```
w1 w2 w3
w1 w1 w3
w1 w3 w3
```

Split each word at the space

```
flatMap(lambda:x x.split(" ")
```

```
[w1
w2
w3
w1
w1
w3
w1
w3
w3]
```

A flapMap is a transformation each RDD element using a function that could return multiple elements to new RDD. Note the RDD size went from 3 to 9 rows

For each word, create a key (word) and count (1)

```
map(lambda x: (x,1))
```

```
[(w1,1)
(w2,1)
(w3,1)
(w1,1)
(w1,1)
(w3,1)
(w1,1)
(w3,1)
(w3,1)]
```

Use a map to transform each RDD element from a word to a Key/Value (word, 1)

Count all words by their key

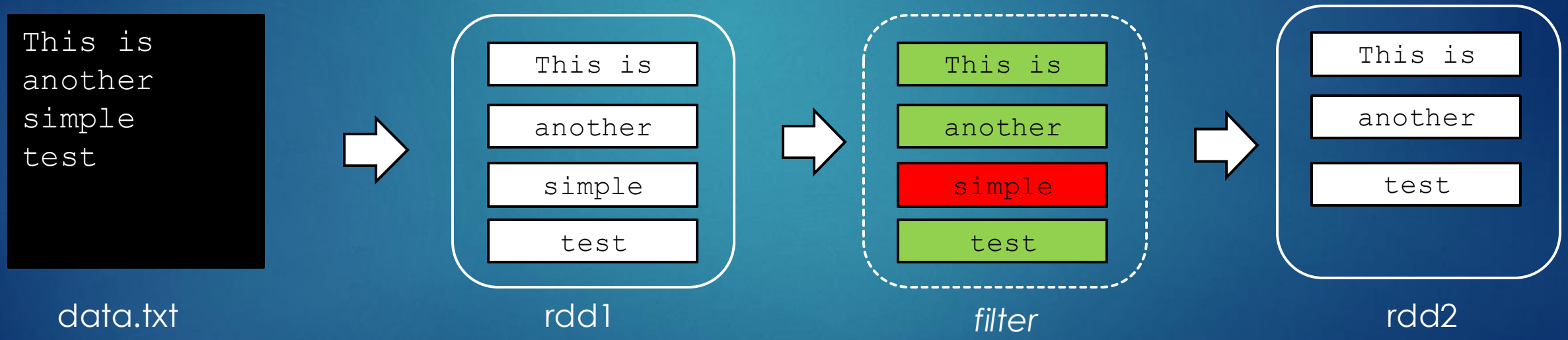
```
reduceByKey(lambda x,y: x+y)
```

```
[(w1,4)
(w2,1)
(w3,4)]
```

A reduceByKey is a transformation which operate on pairRDD (Key/Value). Works like a reduce but stores the result for each unique key

Spark – Example #3

- What if we wanted to just remove the line that contains "simple"?
- One other useful operation is **filter**, which is an operation that returns a new RDD formed by selecting those elements of the source on which the function returns true. Anything that is false is not returned.



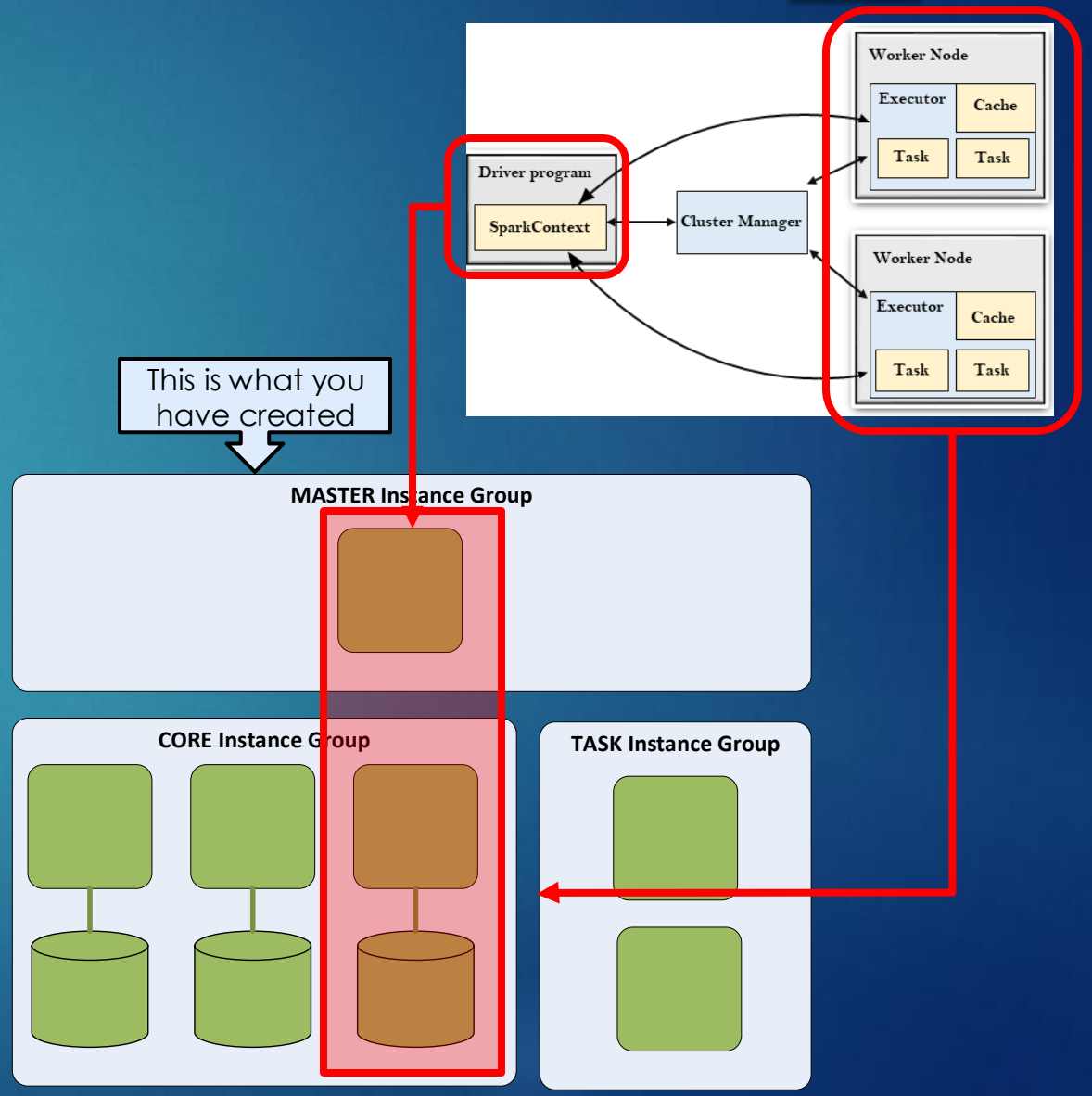

```
rdd1 = sc.textFile("data.txt")        // SC = Spark Context
rdd2 = rdd1.filter(lambda x: "simple" not in x)
```

Hadoop on AWS - EMR

- Amazon EMR (Elastic MapReduce) provides a fully managed Hadoop framework
- Comes preloaded with popular ecosystem tools like Pig, Hive, and Spark
- Integrates seamlessly with AWS services such as S3, DynamoDB, and CloudFormation.
- Supports HDFS as well as other AWS storage options.
- Auto-scaling adjusts cluster size automatically based on utilization — with pay-as-you-go pricing
- Simply bring your data and applications, and EMR handles the rest

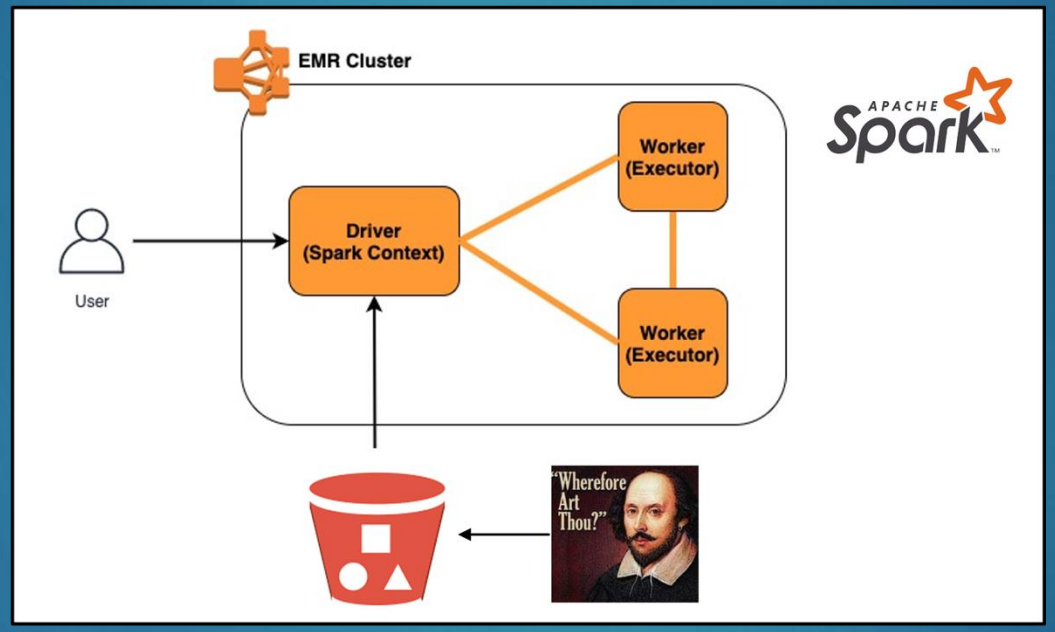
EMR – Master, Core, and Task nodes

- Master node
  - A node (**NameNode**) that manages the cluster by running software components to coordinate the distribution of data and tasks among other nodes for processing
  - The master node tracks the status of tasks and monitors the health of the cluster
- Core node
  - A node with software components that run tasks and store data in the Hadoop Distributed File System (HDFS) on your cluster (**DataNode**)
- Task node
  - An optional node with software components that only runs tasks and does not store data in HDFS (no **DataNode**)



Spark Activity

- For this activity, you will run a Spark wordcount program on your new cluster to count all the words for all of Shakespeare's publications



- Located in Assignments > Activity – Wordcount with Spark
- There are 3 programming tasks plus an opportunity for a **bonus point**